

## DEV2 – JAVL – Laboratoire Java

## TD 12 – Enumerations et records

## Résumé

Dans ce TD, nous explorons ensemble deux catégories de classes particulières : les énumérations (**enum**) et les records (**record**). Les énumérations définissent des types de données qui possèdent un nombre limité de valeurs. Les records, nous permettent de créer des agrégations de valeurs.

## Table des matières

|                                                                     |          |
|---------------------------------------------------------------------|----------|
| <b>1 Enumerations</b>                                               | <b>1</b> |
| 1.1 La classe <code>enum</code>                                     | 1        |
| 1.2 Constructeurs, méthodes et attributs                            | 3        |
| 1.3 Les méthodes <code>values()</code> et <code>valueOf()</code>    | 4        |
| 1.4 Les méthodes <code>ordinal()</code> et <code>compareTo()</code> | 4        |
| <b>2 Records</b>                                                    | <b>5</b> |
| 2.1 La classe <code>record</code>                                   | 5        |

**1 Enumerations**

Nous avons déjà rencontré les énumérations lors du TD 4. Une énumération est une classe particulière qui ne peut prendre que quelques valeurs. Ces valeurs sont fixées lors de l'écriture de la classe. Tout simplement, une énumération crée un nouveau type de données contenant une liste de constantes nommées.

**1.1 La classe `enum`**

On crée une énumération à l'aide du mot-clé `enum`. Par exemple, on pourrait définir une énumération de type `Day` pour représenter dans un programme les différents jours de la semaine par leur nom plutôt que par leur numéro.

```
enum Day {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,  
    ↪ SUNDAY  
}
```

Ici, les membres<sup>1</sup> de l'énumération (`MONDAY`, `TUESDAY`, ..., `SUNDAY`) sont des constantes de type `Day` et sont déclarés implicitement comme membres `public` et `static final`.

1. Par convention, les noms des constantes d'une énumération sont écrits avec des lettres majuscules.

Pour utiliser cette énumération, on écrira par exemple dans un programme :

```
Day d1; // déclaration d'une référence de type Day
d1 = Day.MONDAY; // assignation de la constante MONDAY à d1
Day d2 = Day.TUESDAY;
```

#### class vs enum

À la différence des autres types de classes, on ne crée pas un nouvel objet à l'aide de l'opérateur `new` ! De plus, une énumération ne peut pas être une super-classe ou hériter d'une autre classe. Excepté la classe `java.lang.Enum` dont elle hérite automatiquement.

```
Day d3 = new Day(); // ERROR
```

Pour comparer 2 constantes d'une même énumération, on utilise l'opérateur de comparaison `==` :

```
if ( d1 == Day.MONDAY ) // ...
```

Les valeurs d'une énumération peuvent aussi être utilisées dans une instruction `switch` :

```
// Utilisation d'une enum dans un switch
switch(d1) {
    case MONDAY :
        // ...
    case TUESDAY :
        // ...
```

#### equals vs ==

Dans une énumération, les constantes définies sont des objets du type de l'`enum` et il n'existe qu'une seule instance de chaque constante. C'est la raison pour laquelle, il est possible d'utiliser l'opérateur de comparaison `==` à la place de la méthode `equals` pour comparer des références d'objets du type de l'énumération.

#### Exercice 1

### Enum Transport

- ▷ Définissez une énumération `Transport` qui contient les constantes suivantes : AIR-PLANE, BOAT, CAR, TRAIN, TRUCK.
- ▷ Dans une classe `Main`, créez deux objets de type `Transport`, testez l'égalité des deux objets.
- ▷ À l'aide d'une instruction `switch`, affichez le nom d'une instance de la classe `Transport`. Pour info, les classes `enum` héritent de la classe `Enum` qui redéfinit la méthode `toString` de la classe `Object`.

## 1.2 Constructeurs, méthodes et attributs

Comme toutes les classes, une énumération peut définir des attributs, des constructeurs et des méthodes. À titre d'exemple, reprenons la classe `Day`. Nous allons définir :

- ▷ un constructeur `Day(boolean workingDay)` ;
- ▷ un attribut `private boolean workingDay` qui définit si une constante de type `Day` est un jour de travail ;
- ▷ un **accesseur** pour cet attribut.

```
public enum Day {  
    MONDAY(true), TUESDAY(true), WEDNESDAY(true), THURSDAY(true),  
    FRIDAY(true), SATURDAY(false), SUNDAY(false);  
  
    private boolean workingDay;  
  
    Day(boolean workingDay) {  
        this.workingDay = workingDay;  
    }  
  
    public boolean isWorkingDay() {  
        return workingDay;  
    }  
}
```

### Remarques

- ▷ Les constantes définies dans une énumération sont des objets du type de l'enum.
- ▷ Lors de la création d'une constante de type `Day`, un constructeur (par défaut ou celui que vous avez défini) est appelé.
- ▷ Le constructeur ajouté est privé par défaut sans que l'on ne doive le préciser.
- ▷ Si vous définissez vos propres constructeurs, attributs et méthodes, la liste des constantes de l'énumération doit se terminer par un point virgule.
- ▷ Si votre constructeur exige des paramètres, les constantes doivent être "initialisées" à l'aide des valeurs de ces paramètres (voir exemple ci-dessus).

### Exercice 2

## Vitesse d'un transport

- ▷ Modifiez votre classe `Transport` pour inclure :
  - ▷ un attribut `private int speed` qui définit la vitesse d'un transport ;
  - ▷ un constructeur `Transport(int speed)` ;
  - ▷ un **getter** pour l'attribut `speed`.
- ▷ Dans la classe `Main`, affichez la vitesse de chaque transport.

Vous veillerez à utiliser des valeurs de vitesse réalistes, exprimées en km/h, pour chaque transport.

### 1.3 Les méthodes `values()` et `valueOf()`

Les énumérations possèdent deux méthodes prédéfinies (vous ne devez pas les écrire!) :

- ▷ `public static enum-type[] values()`
- ▷ `public static enum-type valueOf(String str)`

La première, `values()`, retourne un tableau contenant les valeurs de l'énumération. La deuxième, `valueOf(String str)` retourne la valeur de l'énumération correspondant à la chaîne de caractères `str`. Par exemple, le code suivant :

```
public enum Day {  
    MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY,  
    ↪ SUNDAY  
}  
  
public class EnumExemple {  
    public static void main(String[] args) {  
        Day[] days = Day.values();  
        System.out.println("The days of the week :");  
        for(Day day : days) {  
            System.out.println(day);  
        }  
        System.out.println();  
        Day day = Day.valueOf("WEDNESDAY");  
        System.out.println("Today : "+day);  
    }  
}
```

produira la sortie :

```
The days of the week :  
MONDAY  
TUESDAY  
WEDNESDAY  
THURSDAY  
FRIDAY  
SATURDAY  
SUNDAY  
  
Today : WEDNESDAY
```

#### Exercice 3 Affichez les différents moyens de transport

- ▷ Dans la classe `Main` définie plus haut, affichez à l'aide de la méthode `values()`, les différents moyens de transport.
- ▷ À l'aide de la méthode `valueOf(String str)` affichez la valeur de l'énumération correspondant à la chaîne de caractères `"BOAT"`.

### 1.4 Les méthodes `ordinal()` et `compareTo()`

Nous avons mentionné plus haut que toutes les énumérations sont héritières de la classe `java.lang.Enum`. En particulier, elles héritent des deux méthodes suivantes :

- ▷ `final int ordinal()`
- ▷ `final int compareTo(enum-type e)`

La méthode `ordinal()` retourne la position d'une constante dans la liste des constantes d'une énumération en commençant par 0. Par exemple, dans l'énumération `Day` définie plus haut, `Day.MONDAY.ordinal()` retournera 0 et `Day.WEDNESDAY.ordinal()` retournera 2.

La méthode `compareTo(enum-type e)` compare la position de l'objet appelant et de l'objet passé en paramètre dans la liste des constantes d'une énumération. La méthode retourne :

- ▷ une valeur négative, si l'objet appelant se trouve avant l'objet passé en paramètre dans la liste des constantes ;
- ▷ 0 si ils ont la même valeur ordinale ;
- ▷ une valeur positive, si l'objet appelant se trouve après l'objet passé en paramètre dans la liste des constantes.

Par exemple l'appel :

- ▷ `Day.MONDAY.compareTo(Day.TUESDAY)` retournera une valeur négative ;
- ▷ `Day.MONDAY.compareTo(Day.MONDAY)` retournera 0
- ▷ `Day.SATURDAY.compareTo(Day.WEDNESDAY)` retournera une valeur positive.

#### Exercice 4

### Valeur ordinale

- ▷ Dans votre classe `Main`, affichez la valeur ordinale de chaque transport.
- ▷ Testez la méthode `compareTo`.

## 2 Records

Les records sont utilisés pour regrouper des données reliées entre-elles derrière une même abstraction. Par exemple un compte bancaire est composé du nom du client, du numéro du compte et du solde. Les coordonnées d'un point dans une application graphique de ses coordonnées x et y. Bien entendu, tout cela était déjà possible avec les classes que vous avez définies lors des TD précédents. Quoi de neuf ? Les classes `record`, apparues avec JDK 16, facilitent leur développement.

### 2.1 La classe `record`

Pour illustrer notre propos, définissons une classe `record` de type `Product` qui possède les attributs suivants :

- ▷ `String name`
- ▷ `int id` (un identifiant numérique)
- ▷ `double price`

La définition de la classe `Product` se fait simplement à l'aide du mot-clé `record` de la manière suivante :

```
record Product (String name, int id, double price) { }
```

#### class vs record

- ▷ Un `record` ne peut pas être une super-classe ou hériter d'une autre classe. Excepté la classe `java.lang.Record` dont elle hérite automatiquement.
- ▷ Il est possible de créer ses propres constructeurs mais ce point ne sera pas abordé dans ce TD.

### Remarques

- ▷ Les attributs apparaissent directement après le nom de la classe, entre parenthèses et séparés par des virgules.
- ▷ Le corps de la classe peut être vide.
- ▷ Le compilateur génère automatiquement :
  - ▷ les attributs `private final` `name`, `id` et `price` ;
  - ▷ des accesseurs `public` qui portent le même nom que les attributs : `name()`, `id()` et `price()` ;
  - ▷ un constructeur canonique `Product(String name, int id, double price)`.
- ▷ Le compilateur redéfinit aussi les méthodes `toString()`, `equals()` et `hashCode()` héritées de la classe `Object`.

Pour instancier un `record`, on utilise l'opérateur `new`. Exemple :

```
Product item = new Product("Chair", 123, 59.5);
```

#### Exercice 5

### record Student

- ▷ Créez un `record` de type `Student`, qui possède les attributs `String lastName`, `String firstName` et `int idNumber`.
- ▷ Dans une classe `Main`, créez une liste de quelques objets de type `Student` et affichez à l'aide d'un `foreach` le nom, prénom et l'identifiant de chaque étudiant de la liste.
- ▷ Testez les méthodes `toString()`, `equals()` et `hashCode()` héritées de la classe `Object`.

#### Exercice 6

### Exercice de réflexion

Quand utiliseriez-vous une classe `enum` ou une classe `record` ?